

计算机网络、第三章: 3、4、7、14、15、20、22、27、32、36、40、43、48、52、56、58

3. 01010011 ~~反码: 10101100~~

① 01100110 ~~反码: 10011001~~

+ 01101000 ~~反码: 10001011~~

10010110 ~~反码: 10101001~~

因此 00101110 , 八比特字节和的反码为 11010001

②. 用1位标记错误, 若得到结果含有1位则说明有1个错误

③. 立端回卷, 对字节对齐或分组方式的影响较小, 便于实现; 同时规定全1正确校验码

④. 1位比特的差错, 一定可以检测出来

⑤. 2位比特的差错不一定, 有可能会被漏检

4. a. 0101100

b. 11011010

c. 01010100

+ 01100101

+ 01100101

01101101

11000001

10011111 反: 00000000

反码: 00111110

反码: 00100000

7. 序列号用于接收端是否收到重复的数据包

rtts (停止-等待) 通过ACK字符表示已经被确认的序号来指定哪个数据包已被确认, 不再需要单独的序号。

14. 在NAK协议中, 数据包几的丢失只有数据包n到达才能发现

因为“偶尔发送数据”, 所以数据的修正需要经过很长时间, 只用NAK并不佳
若“要发送大量数据”, 并且端到端延迟很少包, 则只用NAK的效率会更高。

~~15. 1501011000010000 传输数据~~

~~1501011000010000 1501011000010000~~

~~1501011000010000 1501011000010000~~



$$80/500 \times 10^9 = 0.012 \text{ ms} = 12 \mu\text{s}$$

15. 设窗口大小为 n .

则利用率

$$\frac{0.012n}{50.012} > 90\%$$

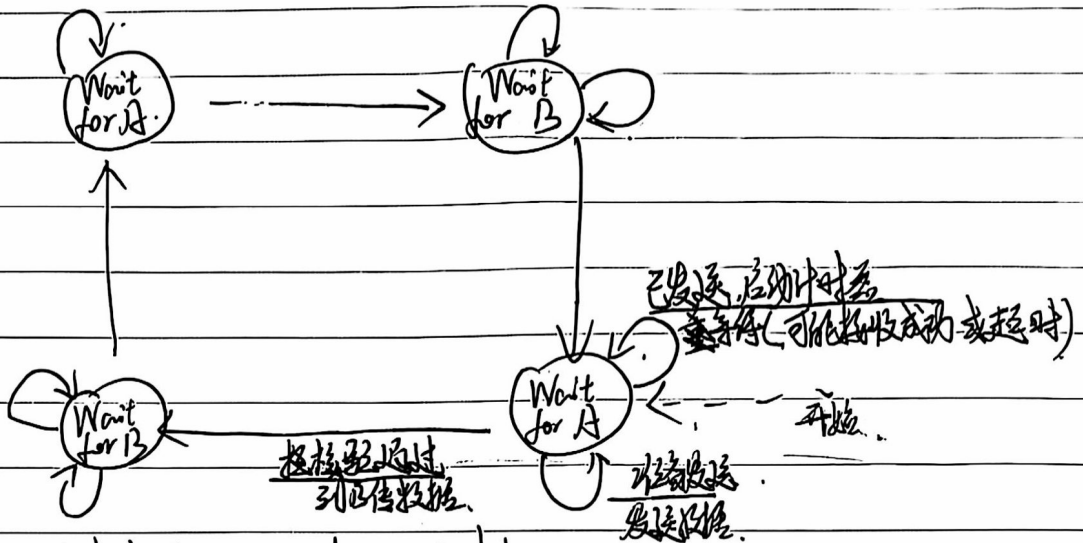
$$\frac{0.012n}{50.012} > 90\%$$

$$n > 2250.9$$

$$n > 2275.9$$

即窗口大小应为 ~~2275~~ 2251

20.



要点: A/B 的数据报用 1 比特位 0/1 来标识

报文格式: ① 数据报 (A/B → C)

表示 DATA 的类型 1 位, 发起源 A/B 1 位, 序号的位, 校验和 16 位, 1

② ACK 报 (C → A/B)

ACK 类型 1 位, 被确认的序号 1 位, 目的地 1 位, 校验和 16 位

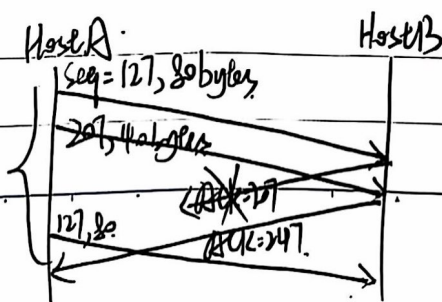
27. ~~a. 序号是 127, 序号是 80, 目的序号是 302~~

a. 序号是 207 (127 + 80), 序号是 302, 目的序号是 80

b. 序号是 207, 序号是 80, 目的序号是 302

c. 此及时乱序, 序号是 127

d.



$$32. (a) \text{Estimated RTT} = \alpha \text{Sample RTT}_1 + (1-\alpha) \alpha \text{Sample RTT}_2 + (1-\alpha)^2 \alpha \text{Sample RTT}_3 + (1-\alpha)^3 \alpha \text{Sample RTT}_4$$

$$(b) \text{Estimated RTT} = \alpha \sum_{j=1}^n (1-\alpha)^{j-1} \text{Sample RTT}_j + (1-\alpha)^n \alpha \text{Sample RTT}_n$$

$$(c) \text{Estimated RTT} = \frac{1}{\alpha} \sum_{j=1}^n \alpha^j \text{Sample RTT}_j$$

36. 若快速重传, 则会降低确认机制的效率

采用3个冗余ACK可以一次确认最多3个数据, 且重传的代价较小, 前面部分

正确的数据不需要重传

40. (a) [1, 6], [23, 26]

(b) [6, 16], [17, 22]

(c) 三次重传ACK

(d) 超时控制

(e) 32

(f) $42/2 = 21$

(g) $\lfloor 29/2 \rfloor = 14$

(h) 第7轮

(i) 窗口长度为4, ssthresh为7

(j) 21, 1

(k) 52个

43. ①不能保证TCP强制把应用层的数据以S速率发出网络

②TCP的拥塞控制也会限制出网速率, 保证不超过网络能力

③可以使用socket或在应用层实现精确控制



48. (a) $W \times 1500 \times 8 / 0.15 = 10 \times 10^9$

则 $W = 125000 \text{ segs}$

b. $0.75 \times W = 93750 \text{ segs}$

平均: $93750 \times 1500 \times 8 / 0.1 = 7.5 \text{ Gbps}$

c. $93750 / 2 \times 0.15 / 60 = 117 \text{ mins}$

52. a. ① t_{200ms} C_1 提升为 200 报文/s

C_2 提升为 100 报文/s

窗口减半

② t_{250ms} $C_1 = 10/2 = 5$

$C_2 = 10/2 = 5$

③ t_{100ms} $C_1 = 5$, $C_2 = 5$

窗口减半

最后窗口为 1 ($1/0.05 + 1/0.1 = 30$ 报文/s, 网络等于网络容量)

即 C_1 , C_2 窗口为 1

b. 在 AMID 中, 长期吞吐量与 RTT 成反比

C_1 吞吐量: C_2 吞吐量 = 2:1

因此长期运行后并非相同, C_1 将发送更多。



56. 优点: ①快速启动 ②保持历史信息 ③减少延迟

缺点: ①网络状态不能确定 ②可能堵塞

建议: 重置 cwnd, 保持 ssthresh, 重新慢启动

这样既考虑网络状况变化, 又不会对空闲期过度惩罚

58. (a) $RTT + 14S/R (= RTT + RTT + S/R + RTT + S/R + RTT + 12S/R)$

(b) $5RTT + 11S/R (= RTT + RTT + S/R + RTT + S/R + RTT + S/R + RTT + 8S/R)$

(c) $3RTT + 15S/R (= RTT + RTT + S/R + RTT + 14S/R)$

22. a. 接收方希望下一个数据包序号为 k , 说明此时刚接收完 $k-1$, 且返回 Ack (前面漏了)

① 若 Ack 接收到, 则发送方窗口序号为 $[k, k+N-1]$

② 若 Ack 未被接收到, 则重传, 窗口为 $[k-N, k-1]$

即窗口为 $[k-N, k]$ 其中 N 为接收方窗口大小

b. 若接收到 1 个或多个, 则为 $[k-N, k-1]$

若都没接收到, 则为 $k-N-1$

即 Ack 所有可能值 $[k-N-1, k-1]$

